SkillHub • Enterprise Backend Architecture Deep Dive

Master Engineering Specification: Clean Architecture, Security Lifecycle, Entity Framework Core 8, Dependency Injection & RESTful Orchestration

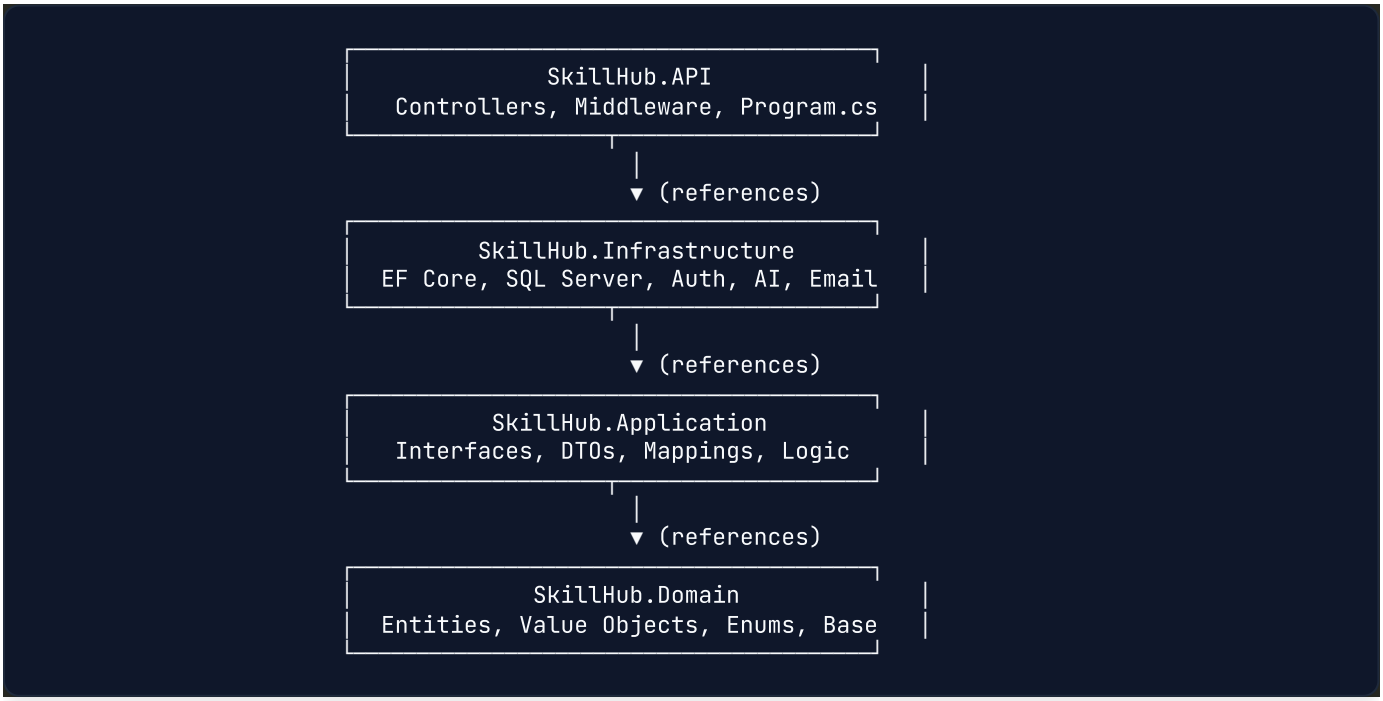
ASP.NET Core 8 Web API | Clean Architecture (Onion) | JWT & Token Rotation | Hybrid RBAC + PBAC | EF Core 8 (19 DbSets) | AutoMapper Projections

SkillHub Backend Architecture & Technical Mastery Guide

Comprehensive Engineering Reference: Clean Architecture, Security, Persistence, Orchestration, and API Design in ASP.NET Core 8

1. Executive Architecture & Clean Architecture Philosophy

SkillHub is an enterprise learning management and assessment platform engineered using **ASP.NET Core 8 Web API** and **Angular 20**. To ensure scalability, testability, and long-term maintainability, the backend strictly adheres to **Clean Architecture** (also known as the Onion / Hexagonal Architecture pattern).



Why Clean Architecture Over Traditional 3-Tier Architecture?

In traditional N-Tier architecture (UI -> Business Logic -> Data Access), the entire system depends directly on the database layer. Database schema changes leak upward into business rules, and unit testing requires mocking live database connections or ORM contexts.

Clean Architecture reverses this dependency through the **Dependency Inversion Principle (DIP)**: 1. **The Core is Independent**: The Domain layer contains pure business logic and entity models with **zero external dependencies** (no EF Core, no ASP.NET, no third-party NuGet packages). 2. **Abstractions Belong to the Business**: The Application layer defines contracts (`interfaces`) describing *what* needs to be done. 3. **Details Depend on Abstractions**: The Infrastructure layer implements *how* it is done (e.g. SQL Server via EF Core, Google Gemini via HttpClient, JWT generation). 4. **Interchangeability**: The database engine, AI provider, or presentation framework can be replaced without modifying a single line of business or domain logic.

The Four Layers in the SkillHub Codebase

1. `SkillHub.Domain` (Core Business Objects)

- **Role**: Contains the enterprise entities, aggregates, domain enums, and base entities.
- **Dependencies**: None. Pure .NET standard classes.
- **Key Files & Structures**:
- `BaseEntity.cs` : Common audit fields inherited by all database models: `csharp public abstract class BaseEntity { public long Id { get; set; } public DateTime CreatedAt { get; set; } = DateTime.UtcNow; public DateTime? UpdatedAt { get; set; } public string? CreatedBy { get; set; } public string? UpdatedBy { get; set; } }`
- `Entities/Identity/` : `User.cs` , `Role.cs` , `UserRole.cs` , `Permission.cs` , `RolePermission.cs` , `UserPermission.cs` , `RefreshToken.cs` .
- `Entities/Courses/` : `Course.cs` , `CourseSection.cs` , `CourseLesson.cs` , `LessonResource.cs` , `CourseCategory.cs` , `Enrollment.cs` , `CourseTag.cs` , `CourseTagMapping.cs` .
- `Entities/Learning/` : `LessonCompletion.cs` , `CourseReview.cs` , `Certificate.cs` .
- `Entities/Assessments/` : `Quiz.cs` , `QuizQuestion.cs` , `QuizOption.cs` , `QuizAttempt.cs` , `QuizAnswer.cs` .
- `Entities/Support/` : `SupportTicket.cs` , `TicketMessage.cs` .

2. `SkillHub.Application` (Business Orchestration Contracts)

- **Role**: Defines business use cases, interface contracts, Data Transfer Objects (DTOs), mapping profiles, and validation logic.
- **Dependencies**: References `SkillHub.Domain` , AutoMapper, FluentValidation.
- **Key Files & Structures**:

- `Features/*/Interfaces/` : `IAuthService` , `IJwtTokenGenerator` , `ICourseService` , `ICurriculumService` , `ILearningService` , `IQuizService` , `ISupportService` , `IAiChatService` , `IPermissionService` .
- `Features/*/DTOs/` : `CourseDtos.cs` , `LearningDtos.cs` , `QuizDtos.cs` , `SupportDtos.cs` , `AiChatDtos.cs` , `AdminDtos.cs` .
- `Mappings/MappingProfile.cs` : AutoMapper configuration mapping entities to DTOs and vice versa.

3. `SkillHub.Infrastructure` (Data Access, External Services & Persistence)

- **Role**: Implements the interfaces defined in `SkillHub.Application` . Connects to Microsoft SQL Server via EF Core 8, handles cryptographic operations, JWT tokens, BCrypt password hashing, SMTP email dispatch, and Google Gemini AI calls.
- **Dependencies**: References `SkillHub.Application` , `SkillHub.Domain` , Microsoft.EntityFrameworkCore.SqlServer, BCrypt.Net-Next, System.IdentityModel.Tokens.Jwt.
- **Key Files & Structures**:
- `Persistence/Context/ApplicationDbContext.cs` : EF Core database context configuring 19 `DbSet` collections and relational constraints.
- `Authentication/` : `JwtTokenGenerator.cs` , `AuthService.cs` , `PermissionAuthorizationHandler.cs` , `PermissionPolicyProvider.cs` , `PermissionRequirement.cs` .
- `Services/` : Concrete implementations of all application interfaces (`CourseService.cs` , `LearningService.cs` , `QuizService.cs` , `SupportService.cs` , `GeminiAiService.cs` , `PermissionService.cs`).
- `DependencyInjection/ServiceRegistration.cs` : Extension method `AddInfrastructure()` registering all services and options into the DI container.

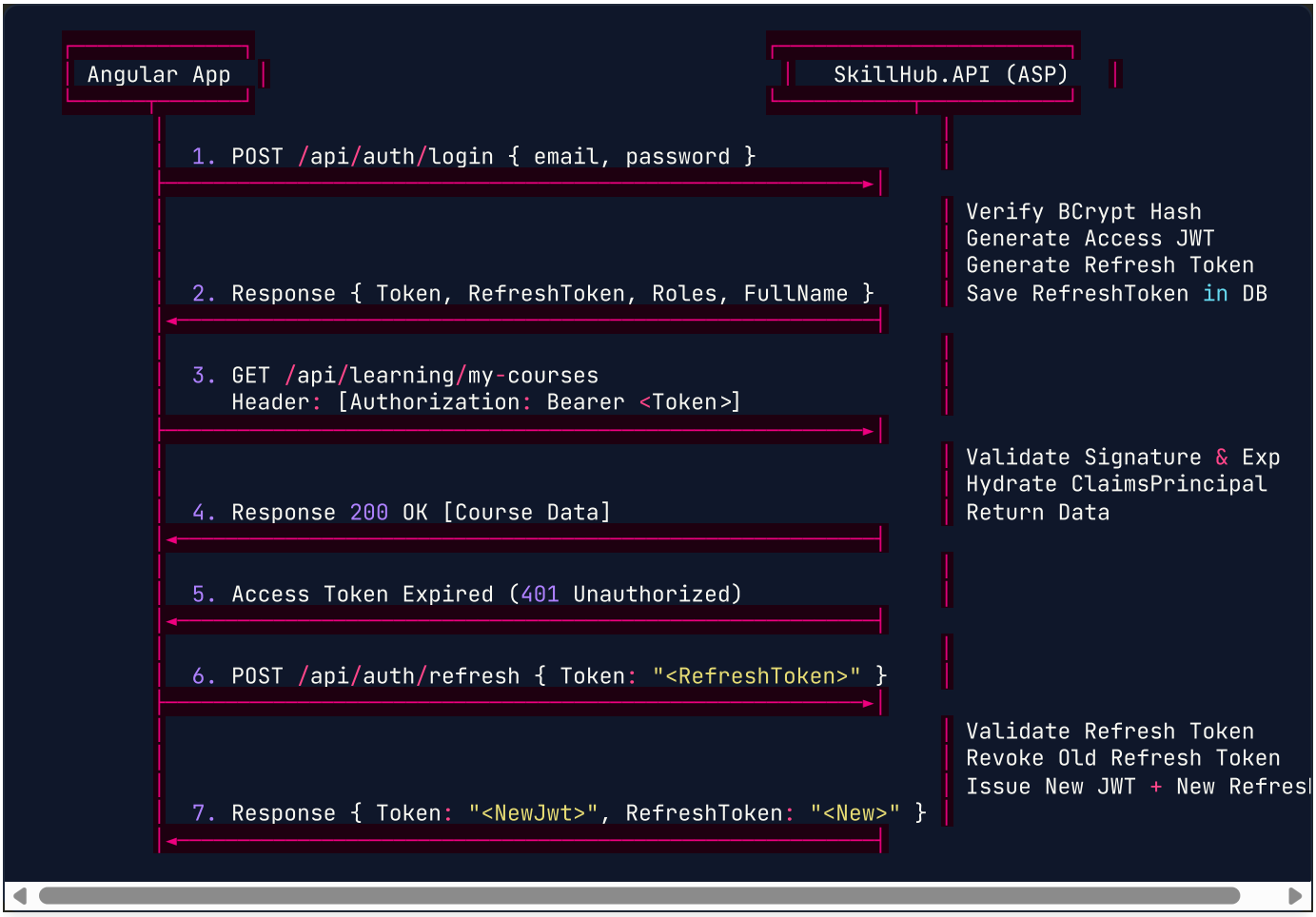
4. `SkillHub.API` (Presentation & HTTP Delivery)

- **Role**: The entry point and composition root of the application. Listens for HTTP requests, applies middleware (CORS, Authentication, Authorization), routes requests to Controllers, and serializes responses to JSON.
- **Dependencies**: References `SkillHub.Infrastructure` and `SkillHub.Application` .
- **Key Files & Structures**:
- `Program.cs` : Configures the ASP.NET Core pipeline, DI registrations, JWT bearer options, CORS policies, Swagger, and startup database migration and seeding.
- `Controllers/` : `AuthController.cs` , `CourseController.cs` , `CurriculumController.cs` , `LearningController.cs` , `QuizzesController.cs` , `SupportController.cs` , `AdminController.cs` , `PermissionsController.cs` , `AiChatController.cs` .

---

2. JSON Web Token (JWT) Authentication & Security Lifecycle

Authentication in modern single-page applications (SPA) must be **stateless**. Traditional cookie-based sessions require server-side memory or distributed session caches (like Redis) to store session IDs, which inhibits horizontal scaling. SkillHub implements stateless authentication using **JSON Web Tokens (RFC 7519)** paired with **database-persisted refresh token rotation**.



Anatomy of a JSON Web Token

A JWT is a compact, URL-safe string divided into three distinct segments separated by periods (`.`):
$$\text{JWT} = \text{Base64Url}(\text{Header}) \,.\, \text{Base64Url}(\text{Payload}) \,.\, \text{Signature}$$

1. Header

Identifies the token type and the cryptographic signing algorithm:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload (Claims)

Contains verifiable identity assertions about the user and token metadata:

```
{
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/nameidentifier": "42",
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/emailaddress": "alex.chen@skillhub.com",
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/name": "Alex Chen",
  "http://schemas.microsoft.com/ws/2008/06/identity/claims/role": "Instructor",
  "iss": "SkillHubAuthServer",
  "aud": "SkillHubClients",
  "exp": 1792420000,
  "nbf": 1792416400
}
```


3. Signature

Prevents tampering. The signature is computed over the base64-encoded header and payload using a secret key known only to the API server: $$\text{Signature} = \text{HMAC-SHA256}\Big(\text{Base64Url}(\text{Header}) + \text{"."} + \text{Base64Url}(\text{Payload}),\; \text{SecretKey}\Big)$$

> 📌 **IMPORTANT**
>
> **Tamper Resistance**: If an attacker intercepts the token and changes `"role": "Student"` to `"role": "SuperAdmin"`, the base64 payload changes. When the server validates the incoming token, recalculating the HMAC-SHA256 signature produces a hash that does **not** match the signature attached to the token. The server immediately rejects the request with HTTP 401 Unauthorized.

Concrete Implementation in SkillHub

1. Token Generation: `JwtTokenGenerator.cs`

Located in `SkillHub.Infrastructure/Authentication/JwtTokenGenerator.cs`:

```
public class JwtTokenGenerator : IJwtTokenGenerator
{
    private readonly JwtSettings _jwtSettings;

    public JwtTokenGenerator(IOptions<JwtSettings> jwtSettings)
    {
        _jwtSettings = jwtSettings.Value;
    }

    public string GenerateToken(User user)
    {
        // Step 1: Assemble Claims
        var claims = new List<Claim>
        {
            new(ClaimTypes.NameIdentifier, user.Id.ToString()),
            new(ClaimTypes.Email, user.Email),
            new(ClaimTypes.Name, $"{user.FirstName} {user.LastName}")
        };

        // Step 2: Append Role Claims dynamically
        foreach (var userRole in user.UserRoles)
        {
            if (userRole.Role ≠ null)
            {
                claims.Add(new Claim(ClaimTypes.Role, userRole.Role.Name));
            }
        }

        // Step 3: Create Symmetric Key from UTF8 Secret
        var key = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(_jwtSettings.Key));

        // Step 4: Define Signing Credentials using HMAC-SHA256
        var credentials = new SigningCredentials(
            key,
            SecurityAlgorithms.HmacSha256);

        // Step 5: Construct the Token Object
        var token = new JwtSecurityToken(
            issuer: _jwtSettings.Issuer,
            audience: _jwtSettings.Audience,
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(_jwtSettings.DurationInMinutes),
            signingCredentials: credentials);

        // Step 6: Serialize to Base64Url string
        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

2. Refresh Token Rotation: `AuthService.cs`

Access tokens are deliberately given a short lifespan (e.g. 60 minutes) to minimize the attack window if stolen. When they expire, the Angular client presents a long-lived **Refresh Token** to obtain a fresh access token without forcing the user to re-enter their credentials.

SkillHub implements **Token Rotation**: every time a refresh token is used, it is revoked and replaced with a newly generated refresh token:

```
public async Task<AuthResponseDto> RefreshTokenAsync(string token)
{
    var existingToken = await _context.RefreshTokens
        .Include(rt => rt.User)
            .ThenInclude(u => u.UserRoles)
                .ThenInclude(ur => ur.Role)
        .FirstOrDefaultAsync(rt => rt.Token == token);

    if (existingToken == null)
        throw new Exception("Invalid refresh token.");

    if (!existingToken.IsActive)
        throw new Exception("Refresh token has expired or been revoked.");

    // Rotate: revoke the old token
    existingToken.RevokedAt = DateTime.UtcNow;

    // Issue a brand-new cryptographically secure refresh token
    var newRefreshToken = GenerateRefreshToken(existingToken.UserId);
    existingToken.ReplacedByToken = newRefreshToken.Token;

    await _context.RefreshTokens.AddAsync(newRefreshToken);
    await _context.SaveChangesAsync();

    var user = existingToken.User;
    return new AuthResponseDto
    {
        Email = user.Email,
        FullName = $"{user.FirstName} {user.LastName}",
        Token = _jwtTokenGenerator.GenerateToken(user),
        RefreshToken = newRefreshToken.Token,
        Roles = user.UserRoles.Select(ur => ur.Role.Name).ToList()
    };
}

private RefreshToken GenerateRefreshToken(long userId)
{
    var randomNumber = new byte[64];
    using var rng = RandomNumberGenerator.Create();
    rng.GetBytes(randomNumber);

    return new RefreshToken
    {
        Token = Convert.ToBase64String(randomNumber),
        UserId = userId,
        ExpiresAt = DateTime.UtcNow.AddDays(_jwtSettings.RefreshTokenDurationInDays),
        CreatedAt = DateTime.UtcNow
    };
}
```

3. Pipeline Configuration: `Program.cs`

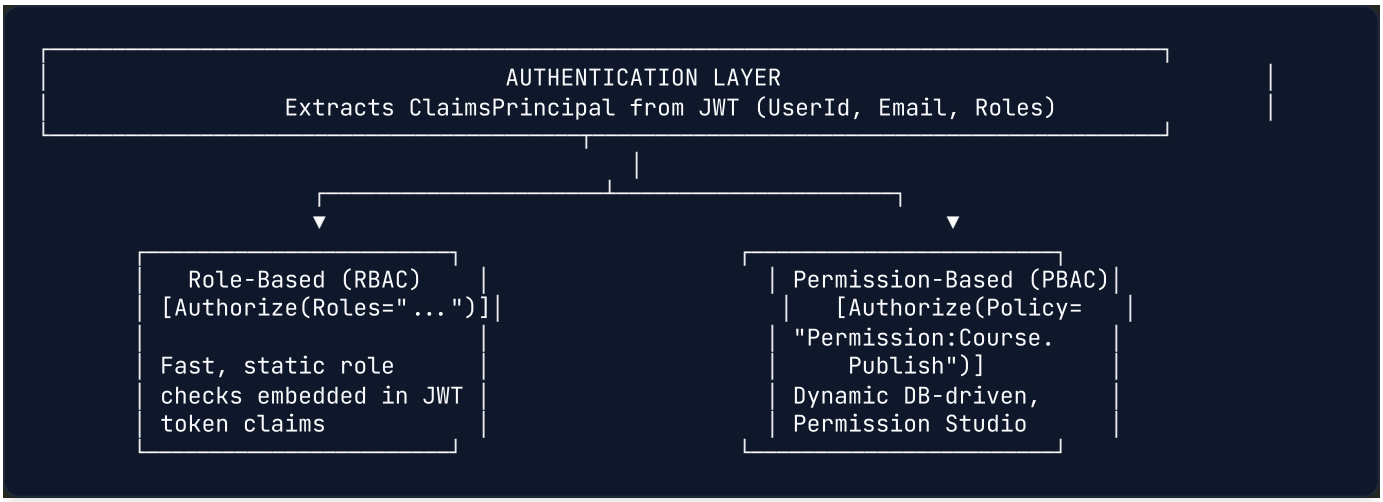
In `SkillHub.API/Program.cs`, ASP.NET Core is configured to authenticate incoming requests via the `JwtBearer` handler:

```
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options ⇒
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["JwtSettings:Issuer"],
            ValidAudience = builder.Configuration["JwtSettings:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["JwtSettings:Key"]!)),
            ClockSkew = TimeSpan.Zero // Eliminates 5-minute leeway on expiration
        };
    });

builder.Services.AddAuthorization();
```

3. Authorization Architecture: RBAC and Dynamic PBAC

Security in SkillHub is dual-layered, providing both high-level role grouping and fine-grained, configurable privileges.



1. Role-Based Access Control (RBAC)

RBAC restricts access based on predefined organizational responsibilities. SkillHub defines 5 core roles: - `Student` : Can browse courses, enroll, track learning progress, submit quiz attempts, view certificates, and submit support tickets. - `Instructor` : Can create and edit curriculum sections/lessons, upload course resources, and view student enrollment statistics. - `SubAdmin` : Operational administrator with delegated management responsibilities (e.g. course approvals). - `SuperAdmin` : Platform master administrator. Complete system oversight, user management, and Permission Studio configuration. - `ProductSupport` : Dedicated helpdesk tier. Exclusively manages, triages, assigns, and resolves customer support tickets.

Implementation in Code:

Controllers and action methods specify role access declaratively:

```
[HttpPost]
[Authorize(Roles = "Instructor")]
public async Task<IActionResult> Create(CreateCourseRequestDto request)
{ ... }

[HttpGet("subadmins")]
[Authorize(Roles = "SuperAdmin")]
public async Task<IActionResult> GetAllSubAdminsWithPermissions()
{ ... }
```

2. Permission-Based Access Control (PBAC)

While RBAC is simple, it suffers from **role explosion** as organizational requirements evolve (e.g. "We need a SubAdmin who can approve courses, but cannot delete users"). PBAC decouples specific actions from static roles.

In SkillHub, PBAC is governed by four database entities: 1. `Permission` : The atomic privilege (e.g., `Course.Create` , `Course.Publish` , `User.Manage` , `Support.Triage`). 2. `RolePermission` : Maps default permissions to a role. 3. `UserPermission` : Allows granting or revoking specific permissions for an individual user, overriding their role defaults. 4. `UserRole` : Associates users with roles.

Dynamic Policy Generation: `PermissionPolicyProvider.cs`

Instead of statically hardcoding hundreds of policy definitions in `Program.cs` , SkillHub implements a custom `IAuthorizationPolicyProvider` :

```
public class PermissionPolicyProvider : IAuthorizationPolicyProvider
{
    private readonly DefaultAuthorizationPolicyProvider _fallbackPolicyProvider;

    public PermissionPolicyProvider(IOptions<AuthorizationOptions> options)
    {
        _fallbackPolicyProvider = new DefaultAuthorizationPolicyProvider(options);
    }

    public Task<AuthorizationPolicy?> GetPolicyAsync(string policyName)
    {
        // Intercept policies prefixed with "Permission:"
        if (policyName.StartsWith(PermissionConstants.PolicyPrefix, StringComparison.OrdinalIgnor
        {
            var permissionName = policyName[PermissionConstants.PolicyPrefix.Length..];

            var policy = new AuthorizationPolicyBuilder()
                .AddRequirements(new PermissionRequirement(permissionName))
                .Build();

            return Task.FromResult<AuthorizationPolicy?>(policy);
        }

        // Fall back to default handler for standard roles
        return _fallbackPolicyProvider.GetPolicyAsync(policyName);
    }

    public Task<AuthorizationPolicy> GetDefaultPolicyAsync() =>
        _fallbackPolicyProvider.GetDefaultPolicyAsync();

    public Task<AuthorizationPolicy?> GetFallbackPolicyAsync() =>
        _fallbackPolicyProvider.GetFallbackPolicyAsync();
}
```

Authorization Evaluation: `PermissionAuthorizationHandler.cs`

The requirement handler queries the database to verify if the authenticated user holds the required permission:

```
public class PermissionAuthorizationHandler : AuthorizationHandler<PermissionRequirement>
{
    private readonly ApplicationDbContext _context;

    public PermissionAuthorizationHandler(ApplicationDbContext context)
    {
        _context = context;
    }

    protected override async Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        PermissionRequirement requirement)
    {
        var userIdClaim = context.User.FindFirst(ClaimTypes.NameIdentifier);
        if (userIdClaim == null || !long.TryParse(userIdClaim.Value, out var userId))
            return;

        // Traverse: User → Roles → RolePermissions → Permission Name
        var hasPermission = await _context.UserRoles
            .Where(ur => ur.UserId == userId)
            .SelectMany(ur => ur.Role.RolePermissions)
            .AnyAsync(rp => rp.Permission.Name == requirement.Permission);

        if (hasPermission)
        {
            context.Succeed(requirement);
        }
    }
}
```

RBAC vs PBAC Comparison

DIMENSION	ROLE-BASED ACCESS CONTROL (RBAC)	PERMISSION-BASED ACCESS CONTROL (PBAC)
Granularity	Coarse-grained (User is `Instructor`)	Fine-grained (User can `Course.UploadVideo`)
Coupling	High: Code references specific role names	Low: Code references business capabilities
Flexibility	Static; changing role rules requires redeployment	Dynamic; permissions toggled live via Permission Studio
Execution Cost	Negligible; claims read directly from JWT	Indexed database query via `PermissionAuthorizationHandler`

4. Entity Framework Core (EF Core) 8 & Persistence Architecture

Entity Framework Core is Microsoft's modern, lightweight, extensible **Object-Relational Mapper (ORM)**. It enables .NET developers to interact with relational databases using strongly typed C# objects, translating LINQ expressions into optimized SQL statements.

`ApplicationDbContext` : The Persistence Heart

`ApplicationDbContext` (in `SkillHub.Infrastructure/Persistence/Context/ApplicationDbContext.cs`) manages database connections, tracks entity changes, and maps 19 distinct `DbSet<T>` entities:

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options) { }

    // 1. Identity & Security (7 DbSets)
    public DbSet<User> Users => Set<User>();
    public DbSet<Role> Roles => Set<Role>();
    public DbSet<UserRole> UserRoles => Set<UserRole>();
    public DbSet<Permission> Permissions => Set<Permission>();
    public DbSet<RolePermission> RolePermissions => Set<RolePermission>();
    public DbSet<UserPermission> UserPermissions => Set<UserPermission>();
    public DbSet<RefreshToken> RefreshTokens => Set<RefreshToken>();

    // 2. Courses & Curriculum (8 DbSets)
    public DbSet<Course> Courses => Set<Course>();
    public DbSet<Enrollment> Enrollments => Set<Enrollment>();
    public DbSet<CourseCategory> CourseCategories => Set<CourseCategory>();
    public DbSet<CourseSection> CourseSections => Set<CourseSection>();
    public DbSet<CourseLesson> CourseLessons => Set<CourseLesson>();
    public DbSet<LessonResource> LessonResources => Set<LessonResource>();
    public DbSet<CourseTag> CourseTags => Set<CourseTag>();
    public DbSet<CourseTagMapping> CourseTagMappings => Set<CourseTagMapping>();

    // 3. Learning & Certification (3 DbSets)
    public DbSet<LessonCompletion> LessonCompletions => Set<LessonCompletion>();
    public DbSet<CourseReview> CourseReviews => Set<CourseReview>();
    public DbSet<Certificate> Certificates => Set<Certificate>();

    // 4. Assessments & Quizzes (5 DbSets)
    public DbSet<Quiz> Quizzes => Set<Quiz>();
    public DbSet<QuizQuestion> QuizQuestions => Set<QuizQuestion>();
    public DbSet<QuizOption> QuizOptions => Set<QuizOption>();
    public DbSet<QuizAttempt> QuizAttempts => Set<QuizAttempt>();
    public DbSet<QuizAnswer> QuizAnswers => Set<QuizAnswer>();

    // 5. Support & Helpdesk (2 DbSets)
    public DbSet<SupportTicket> SupportTickets => Set<SupportTicket>();
    public DbSet<TicketMessage> TicketMessages => Set<TicketMessage>();
```

Fluent API Configuration & Defensive Constraints

Rather than polluting Domain entities with EF Core attributes, SkillHub isolates relational configurations using the Fluent API in `OnModelCreating`:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(ApplicationDbContext).Assembly);

    // SupportTicket Relational Constraints
    modelBuilder.Entity<SupportTicket>(b =>
    {
        b.HasIndex(t => t.TicketNumber).IsUnique();
        b.Property(t => t.TicketNumber).HasMaxLength(50).IsRequired();
        b.Property(t => t.Subject).HasMaxLength(250).IsRequired();

        // Defensive Deletion Rule: Restrict
        b.HasOne(t => t.User)
            .WithMany()
            .HasForeignKey(t => t.UserId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasOne(t => t.AssignedAdmin)
            .WithMany()
            .HasForeignKey(t => t.AssignedAdminId)
            .OnDelete(DeleteBehavior.Restrict);

        // Cascading Deletion Rule: Cascade messages when ticket is purged
        b.HasMany(t => t.Messages)
            .WithOne(m => m.Ticket)
            .HasForeignKey(m => m.TicketId)
            .OnDelete(DeleteBehavior.Cascade);
    });
}
```

> 💡 **TIP**
>
> **Why `DeleteBehavior.Restrict` is Critical**: In enterprise systems, deleting a user must **never** cause a cascading delete of financial records, audit logs, student certificates, or support tickets. By specifying `DeleteBehavior.Restrict`, SQL Server enforces foreign key integrity by throwing an error if someone attempts to delete a user with active tickets or enrollments.

Migrations, Change Tracking, and Query Optimization

1. Migrations Lifecycle

EF Core maintains a snapshot of the database schema in `ApplicationDbContextModelSnapshot.cs`. When schema modifications occur: 1. `dotnet ef migrations add <MigrationName>`: Analyzes entity changes and creates an incremental C# migration script. 2. `dotnet ef database update`: Applies pending migrations to SQL Server and logs them in the `__EFMigrationsHistory` table. 3. In `Program.cs`, SkillHub automatically runs `await context.Database.MigrateAsync();` on application startup to ensure database parity in production environments.

2. Change Tracking Engine

When entities are queried without `AsNoTracking()`, EF Core tracks their original property values in an internal snapshot: - `EntityState.Unchanged`: Fetched from database, not modified. - `EntityState.Modified`: One or more properties changed. - `EntityState.Added`: New entity passed to `AddAsync()`. - `EntityState.Deleted`: Entity passed to `Remove()`.

When `await _context.SaveChangesAsync()` is called, EF Core generates an optimized SQL statement that updates **only** the columns that were modified:

```
var course = await _context.Courses.FirstOrDefaultAsync(x => x.Id == request.Id);
_mapper.Map(request, course); // Updates title, description
await _context.SaveChangesAsync(); // Generates: UPDATE Courses SET Title = @p0, Description = @p
```

3. AsNoTracking & Projection Queries

For read-only operations (e.g. browsing the course catalog or viewing analytics), change tracking is overhead. SkillHub leverages `.ProjectTo<T>()` and `.AsNoTracking()`:

```
return await _context.Courses
    .Where(c => c.IsPublished)
    .OrderByDescending(c => c.CreatedAt)
    .ProjectTo<CourseListDto>(_mapper.ConfigurationProvider)
    .ToListAsync();
```

This instructs EF Core to generate a SQL query selecting **only the exact columns needed by** `CourseListDto`, omitting heavy unused columns (such as full syllabus text or internal audit properties).

---

5. Interfaces, Services & Dependency Injection (DI)

Why Interfaces Matter in Enterprise Architecture

An **Interface** in C# is an abstract contract that specifies method signatures, properties, and events without providing their implementation. 1. **Decoupling**: Callers (like Controllers) depend on the contract (`ICourseService`), not the concrete implementation (`CourseService`). 2. **Unit Testability**: Test suites can mock `ICourseService` (using libraries like `Moq` or `NSubstitute`) to test controller logic in isolation without touching a database. 3. **Multiple Implementations**: You can swap implementations easily (e.g. `LocalFileService` vs `AwsS3FileService` for `IFileService`).

The Dependency Injection (DI) Principle

Inversion of Control (IoC) dictates that objects should not construct their own dependencies. Instead, dependencies are "injected" at runtime by an IoC container.

The Three Service Lifetimes in ASP.NET Core:

```
┌──────────────────┬──────────────────────────────────────────────────────────────┐
│ Lifetime         │ Description & Behavior                                       │
├──────────────────┼──────────────────────────────────────────────────────────────┤
│ **Transient**    │ Created every single time it is requested.                   │
│ `AddTransient`   │ Best for lightweight, stateless utility operations.          │
├──────────────────┼──────────────────────────────────────────────────────────────┤
│ **Scoped**       │ Created once per HTTP request lifecycle.                     │
│ `AddScoped`      │ Shared across all components during that request, then disposed. │
│                  │ Standard for `ApplicationDbContext`, repositories, and services. │
├──────────────────┼──────────────────────────────────────────────────────────────┤
│ **Singleton**    │ Created once when the application starts; lives forever.     │
│ `AddSingleton`   │ Best for application-wide caches, policy providers, singletons. │
└──────────────────┴──────────────────────────────────────────────────────────────┘
```

> ⚠️ **WARNING**
>
> **Captive Dependency Antipattern**: Never inject a `Scoped` service (such as `ApplicationDbContext`) into a `Singleton` service. Doing so traps the Scoped service inside the Singleton, preventing it from being disposed, exhausting database connection pools, and corrupting DbContext change tracking.

Step-by-Step Code Walkthrough: Course Management

Step 1: Define the Contract (`ICourseService.cs`)

Located in `SkillHub.Application/Features/Courses/Interfaces/ICourseService.cs`:

```
public interface ICourseService
{
    Task<CourseResponseDto> CreateAsync(CreateCourseRequestDto request, long instructorId);
    Task<List<CourseListDto>> GetAllAsync();
    Task<CourseResponseDto?> GetByIdAsync(long id);
    Task<List<CourseListDto>> GetCoursesByInstructorAsync(long instructorId);
    Task UpdateAsync(UpdateCourseRequestDto request, long instructorId);
    Task DeleteAsync(long id, long instructorId);
    Task<PlatformStatsDto> GetPlatformStatsAsync();
}
```

Step 2: Implement the Service (`CourseService.cs`)

Located in `SkillHub.Infrastructure/Services/Courses/CourseService.cs`:

```
public class CourseService : ICourseService
{
    private readonly ApplicationDbContext _context;
    private readonly IMapper _mapper;

    // Constructor Injection of Scoped DbContext and AutoMapper
    public CourseService(ApplicationDbContext context, IMapper mapper)
    {
        _context = context;
        _mapper = mapper;
    }

    public async Task<CourseResponseDto> CreateAsync(
        CreateCourseRequestDto request, long instructorId)
    {
        var course = _mapper.Map<Course>(request);
        course.InstructorId = instructorId;

        await _context.Courses.AddAsync(course);
        await _context.SaveChangesAsync();

        return await GetByIdAsync(course.Id)
               ?? throw new Exception("Course creation failed.");
    }
}
```

Step 3: Wire into DI Container (`ServiceRegistration.cs`)

Located in `SkillHub.Infrastructure/DependencyInjection/ServiceRegistration.cs` :

```
public static IServiceCollection AddInfrastructure(
    this IServiceCollection services,
    IConfiguration configuration)
{
    // Register DbContext as Scoped
    services.AddDbContext<ApplicationDbContext>(options =>
        options.UseSqlServer(configuration.GetConnectionString("DefaultConnection")));

    // Register Business Services as Scoped
    services.AddScoped<ICourseService, CourseService>();
    services.AddScoped<IAuthService, AuthService>();
    services.AddScoped<ILearningService, LearningService>();

    // Register Dynamic Policy Provider as Singleton
    services.AddSingleton<IAuthorizationPolicyProvider, PermissionPolicyProvider>();

    return services;
}
```

Step 4: Inject into Controller (`CourseController.cs`)

Located in `SkillHub.API/Controllers/CourseController.cs` :

```
[ApiController]
[Route("api/[controller]")]
public class CourseController : ControllerBase
{
    private readonly ICourseService _courseService;

    // The ASP.NET Core DI runtime automatically resolves ICourseService
    public CourseController(ICourseService courseService)
    {
        _courseService = courseService;
    }

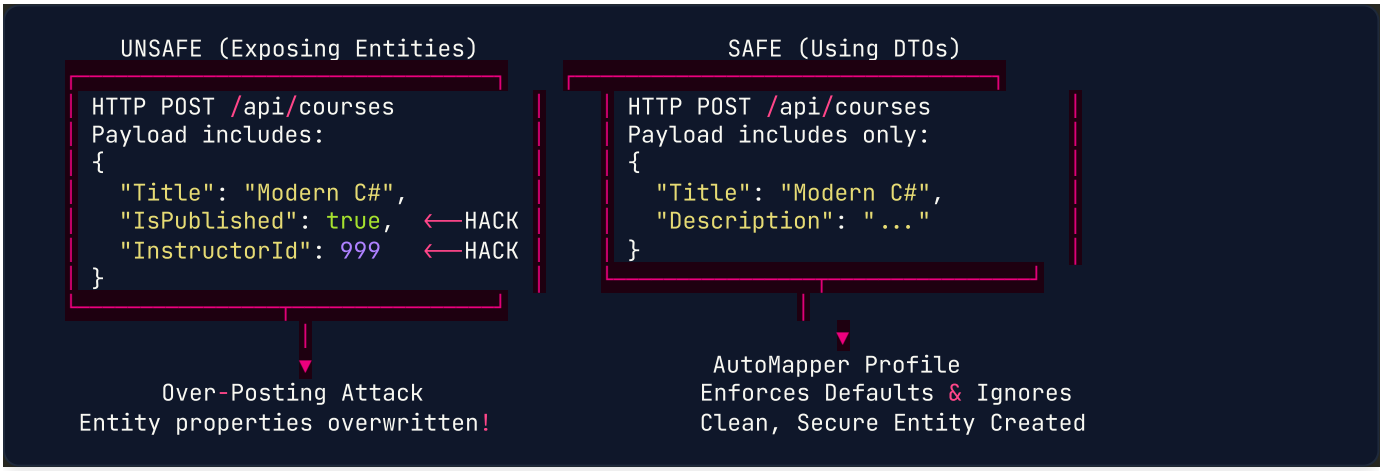
    [HttpPost]
    [Authorize(Roles = "Instructor")]
    public async Task<IActionResult> Create(CreateCourseRequestDto request)
    {
        var userId = long.Parse(User.FindFirst(ClaimTypes.NameIdentifier)!.Value);
        var result = await _courseService.CreateAsync(request, userId);
        return Ok(result);
    }
}
```

---

6. Object Mapping & Data Transfer Objects (DTOs)

Why DTOs are Mandatory

A **Data Transfer Object (DTO)** is a flat, simple object containing only data fields, devoid of business logic. Directly exposing Domain entities over HTTP endpoints is considered a severe architectural defect:



1. **Defense Against Over-Posting (Mass Assignment)**: If an entity has properties like `IsPublished`, `IsVerified`, or `Role`, a malicious user can include `"IsPublished": true` in their JSON payload. A DTO restricts inputs strictly to allowed properties.
2. **Circular Reference Serialization Loops**: A `Course` entity references a list of `CourseSection` entities. Each `CourseSection` references its parent `Course`. When System.Text.Json attempts to serialize this structure, it triggers a `JsonException: A possible object cycle was detected`. Flat DTOs eliminate relational cycles.
3. **Decoupling Public Contract from Database Schema**: Changing a database column name (e.g. `ThumbnailUrl` to `CoverImageStorageKey`) does not break mobile apps or frontend clients when using

DTOs.

4. **Network Performance**: DTOs transmit only the fields the client needs, cutting network bandwidth.

AutoMapper Profile Configuration: `MappingProfile.cs`

AutoMapper automates copying properties between entities and DTOs using reflection and compiled expression trees:

```
public class MappingProfile : Profile
{
    public MappingProfile()
    {
        // 1. Two-way Mapping for Lookup Tables
        CreateMap<CourseCategory, CourseCategoryDto>().ReverseMap();

        // 2. Flattening & Computed Properties
        CreateMap<Course, CourseListDto>()
            .ForMember(d => d.CategoryName,
                       o => o.MapFrom(s => s.Category.Name))
            .ForMember(d => d.InstructorName,
                       o => o.MapFrom(s => $"{s.Instructor.FirstName} {s.Instructor.LastName}"));

        // 3. Flattening Enrollments for Student Dashboard
        CreateMap<Enrollment, CourseListDto>()
            .ForMember(d => d.Id, o => o.MapFrom(s => s.Course.Id))
            .ForMember(d => d.Title, o => o.MapFrom(s => s.Course.Title))
            .ForMember(d => d.ThumbnailUrl, o => o.MapFrom(s => s.Course.ThumbnailUrl))
            .ForMember(d => d.CategoryName, o => o.MapFrom(s => s.Course.Category.Name))
            .ForMember(d => d.InstructorName, o => o.MapFrom(s => $"{s.Course.Instructor.FirstNam

        // 4. Over-Posting Protection on Course Creation
        CreateMap<CreateCourseRequestDto, Course>()
            .ForMember(d => d.IsPublished, o => o.MapFrom(_ => false)); // Force unpublished by d

        // 5. Safe Entity Updates (Ignore Primary Keys)
        CreateMap<UpdateCourseRequestDto, Course>()
            .ForMember(d => d.Id, o => o.Ignore()); // Never overwrite PK during update
    }
}
```

---

7. Repositories & The Persistence Pattern

The Repository Pattern vs EF Core

A common debate in enterprise .NET is whether to wrap EF Core in an explicit `GenericRepository<T>` layer: - **The Case Against Redundant Repositories**: Martin Fowler defines a Repository as an in-memory collection of domain objects and a Unit of Work as managing transactions. In EF Core: - `DbSet<T>` is **already** an implementation of the Repository Pattern. - `DbContext` is **already** an implementation of the Unit of Work Pattern. - Adding generic wrappers like `IRepository<T>` often restricts developers from using powerful EF Core features like `.Include()`, `.ThenInclude()`, and `.ProjectTo<T>()`.

SkillHub employs a **Pragmatic Persistence Architecture**: - Domain-specific business logic is encapsulated within cohesive Services (`CourseService` , `LearningService` , `SupportService`). - Services directly leverage `ApplicationDbContext` for high-performance LINQ queries, change tracking, and projections. - Specialized repositories (like `UserRepository`) are employed where custom identity queries benefit from clean encapsulation.

Command-Query Separation (CQS) inside Services

```
┌──────────────────────────────────────────────────────────────────────────┐
│                            APPLICATION SERVICES                          │
├──────────────────────────────────┬───────────────────────────────────────┤
│      QUERIES (Read Operations)   │      COMMANDS (Write Operations)      │
├──────────────────────────────────┼───────────────────────────────────────┤
│ • Uses `.AsNoTracking()` or `.ProjectTo()`│ • Entity is fetched with Change Tracking │
│ • Zero memory tracking overhead  │ • Properties modified in memory       │
│ • Reads directly to DTOs         │ • `SaveChangesAsync()` executes in atomic │
│ • Example: `GetAllAsync()`, stats queries │   database transaction        │
└──────────────────────────────────┴───────────────────────────────────────┘
```

Explicit Transaction Management:

When multiple operations must succeed or fail as an indivisible unit (e.g., student payment processing and immediate course enrollment unlocking), SkillHub uses database transactions:

```
using var transaction = await _context.Database.BeginTransactionAsync();
try
{
    // Step 1: Create Enrollment
    var enrollment = new Enrollment { UserId = userId, CourseId = courseId };
    await _context.Enrollments.AddAsync(enrollment);

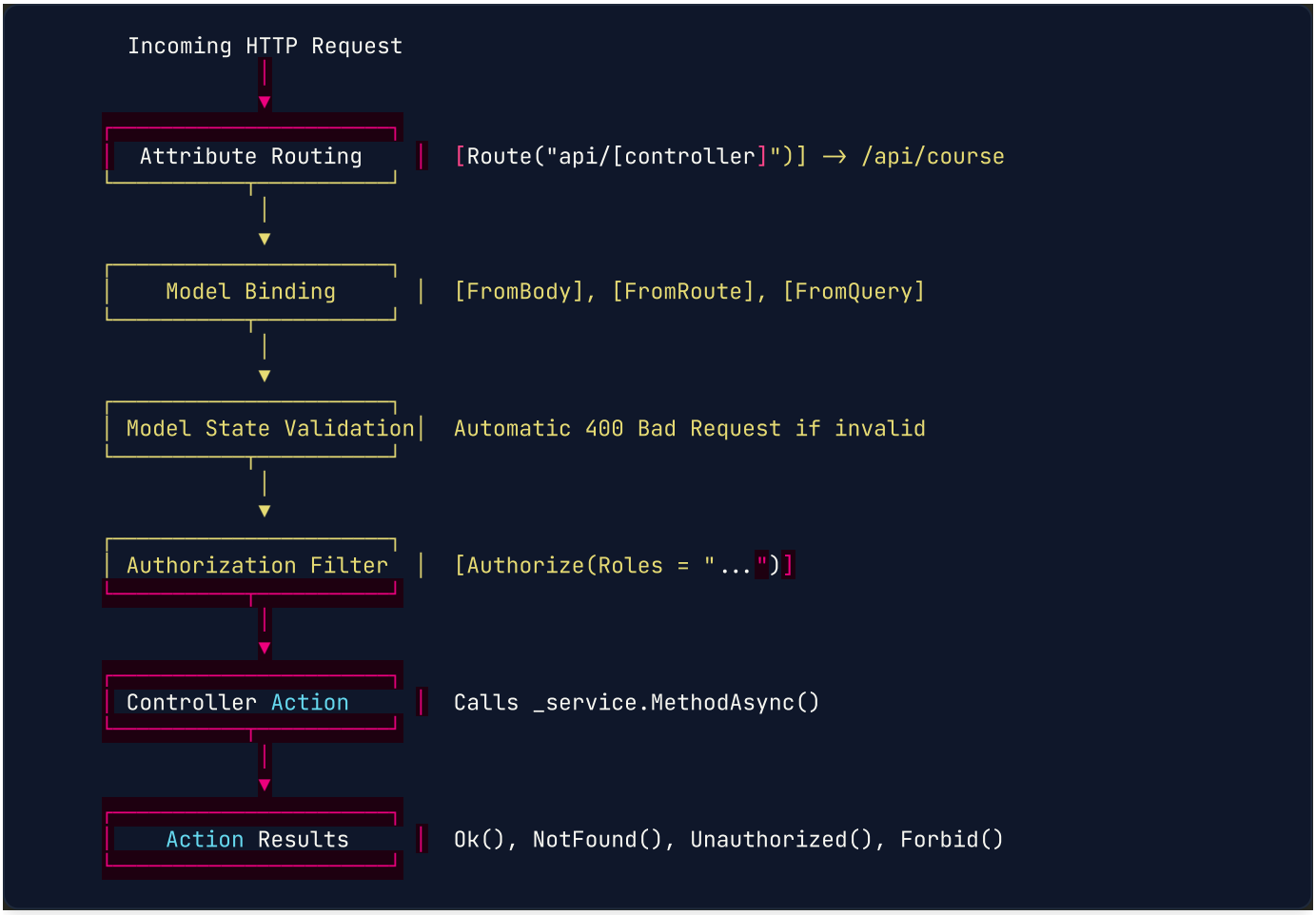
    // Step 2: Record Payment Transaction
    var payment = new PaymentRecord { Enrollment = enrollment, Amount = amount };
    await _context.Payments.AddAsync(payment);

    await _context.SaveChangesAsync();
    await transaction.CommitAsync();
}
catch
{
    await transaction.RollbackAsync();
    throw;
}
```

---

8. Controllers in SkillHub: The Presentation Layer

Controllers in ASP.NET Core are responsible for processing incoming HTTP requests, executing business logic via injected services, and generating appropriate HTTP status codes and responses.



Key Controller Attributes

1. `[ApiController]` : Enables automatic HTTP 400 responses on model validation failure, binds body payloads automatically, and configures problem details for errors.
2. `[Route("api/[controller]")]` : Maps URLs using controller name token replacement (`AuthController` -> `/api/auth`).
3. `ControllerBase` : Provides helper methods (`Ok()` , `BadRequest()` , `NotFound()` , `Forbid()` , `User`) without the overhead of Razor MVC views.

Survey of SkillHub Controllers

CONTROLLER	ROUTE	PURPOSE & KEY RESPONSIBILITIES
`AuthController`	`/api/auth`	User registration, credential verification via BCrypt, JWT generation, refresh token rotation, and 6-digit email password resets.
`CourseController`	`/api/course`	Public course catalog browsing, instructor course creation/updates, homepage platform statistics.
`CurriculumController`	`/api/curriculum`	Section and lesson authoring, video streaming metadata, downloadable reference resource attachments.
`LearningController`	`/api/learning`	Student learning player, real-time lesson completion toggle, course progress percentage, and certificate verification.
`QuizzesController`	`/api/quizzes`	Assessment retrieval, quiz submissions, automated question grading, and certificate 70% mastery validation.
`SupportController`	`/api/support`	Enterprise helpdesk triage, ticket creation, message threads, assignment to admins, and state transitions (`Open` -> `In Progress` -> `Resolved` -> `Closed`).
`AdminController`	`/api/admin`	Platform governance, instructor applications, user status toggles, course publication approvals.
`PermissionsController`	`/api/permissions`	Permission Studio endpoints: role-permission matrix, user permission overrides.
`AiChatController`	`/api/aichat`	Google Gemini AI learning tutor integration with prompt engineering and guest quota rate limiting.

Model Binding and Action Results Deep-Dive

1. Model Binding Sources:

- `[FromBody]` : Binds JSON payload from the HTTP body to a DTO (e.g. `RegisterRequestDto`).
- `[FromRoute]` : Binds parameters directly from URI segments (e.g. `[HttpGet("{id:long}")]`).
- `[FromQuery]` : Binds parameters from the query string (e.g. `/api/courses?page=1&category=Design`).

- `[FromServices]`: Injects a service directly into an action method parameter instead of the constructor.

2. Standard Action Results:

```
[HttpGet("{id:long}")]
public async Task<IActionResult> GetById(long id)
{
    var result = await _courseService.GetByIdAsync(id);
    if (result == null)
    {
        return NotFound(new { message = $"Course with ID {id} not found." }); // HTTP 404
    }
    return Ok(result); // HTTP 200 with serialized JSON
}
```

---

9. Complete End-to-End Request Lifecycle

The diagram below maps the exact execution lifecycle of an authenticated request (e.g. a student completing a lesson):

```
[Angular Client]
       |
       | HTTP POST /api/learning/lessons/105/toggle-completion
       | Headers: [Authorization: Bearer eyJhbGciOi...]
       ▼
[Kestrel Web Server]
       |
       ▼
[CORS Middleware] ── Validates Origin against Allowed Origins
       |
       ▼
[Authentication Middleware] ── Inspects Bearer Token, Validates Signature & Expiration,
       |                       Hydrates HttpContext.User with ClaimsPrincipal
       ▼
[Authorization Middleware] ── Evaluates [Authorize(Roles = "Student")] or Policy Requirements
       |
       ▼
[Endpoint Routing & Model Binder] ── Resolves LearningController, Extracts lessonId=105 from Rou
       |
       ▼
[LearningController.ToggleLessonCompletion(105)]
       | Extracts Student UserId from User.FindFirst(ClaimTypes.NameIdentifier)
       ▼
[ILearningService.ToggleLessonCompletionAsync(userId, 105)]
       |
       ├─► Queries ApplicationDbContext.LessonCompletions
       |   - If completed: Removes entity
       |   - If uncompleted: Adds new LessonCompletion entity
       |
       ├─► Recalculates Progress Percentage:
       |   Progress = (CompletedLessonsCount / TotalLessonsInCourse) * 100
       |
       ├─► Evaluates 70% Mastery Rule:
       |   If Progress == 100% AND All Quizzes ≥ 70%:
       |       Generates and stores new Certificate entity
       |
       └─► Executes await _context.SaveChangesAsync() (Atomic SQL Transaction)
       |
       ▼
[LearningController returns Ok(responseDto)]
       |
       ▼
[Response Pipeline & JSON Serializer]
       |
       ▼
[Angular Client receives 200 OK with Updated Course Progress & Certificate Status]
```

10. Architectural Viva & Technical Defense Cheatsheet

Q1: Why did you choose Clean Architecture instead of traditional N-Tier architecture?

> **Answer**: In traditional N-Tier, the business logic depends directly on the database access layer (ADO.NET or EF Core). Any change in database schema or ORM leaks upward into business rules. In Clean Architecture, we invert this dependency: the core `Domain` and `Application` layers contain pure business rules and interfaces with zero dependencies on frameworks or databases.

> `Infrastructure` and `API` depend inward on `Application`. This makes the application decoupled, highly testable, and adaptable to future technology shifts.

Q2: How does JWT authentication prevent tampering if the token is just Base64Url-encoded text?

> **Answer**: While the header and payload are readable via Base64Url decoding, the third part—the **Signature**—is computed using HMAC-SHA256 over `Base64(Header) + "." + Base64(Payload)` using a private secret key stored securely on the server. If a malicious actor alters any claim in the payload (such as escalating their role), the server recalculates the HMAC signature upon receiving the token. Because the attacker does not possess the secret key, their modified payload will not match the signature, causing the server to reject the request with HTTP 401 Unauthorized.

Q3: What is the difference between RBAC and PBAC in SkillHub, and why did you implement both?

> **Answer**: RBAC (Role-Based Access Control) restricts access based on coarse role categories (`Student`, `Instructor`, `SubAdmin`, `SuperAdmin`, `ProductSupport`). PBAC (Permission-Based Access Control) provides fine-grained authorization by decoupling capabilities from roles into individual permissions (e.g. `Course.Publish`, `Support.Triage`). SkillHub combines both: RBAC handles standard endpoint routing quickly via claims in the JWT token, while our custom `PermissionPolicyProvider` and `PermissionAuthorizationHandler` allow SuperAdmins to dynamically grant or revoke specific permissions in our Permission Studio without redeploying code.

Q4: Explain the difference between `AddScoped`, `AddTransient`, and `AddSingleton`.

> **Answer**: - `AddTransient`: Creates a new instance every time the service is requested. Suitable for lightweight, stateless helper utilities. - `AddScoped`: Creates a single instance per HTTP request lifecycle. All components participating in the request share the same instance. This is essential for `ApplicationDbContext` to ensure a consistent unit of work across services. - `AddSingleton`: Creates a single instance when the application boots, living for the entire process lifetime. Used for thread-safe caches, configuration options, and custom policy providers like `PermissionPolicyProvider`.

Q5: Why do we use AutoMapper and DTOs instead of passing EF Core entities directly to our controllers?

> **Answer**: Directly exposing entities introduces four critical risks: 1. **Mass Assignment / Over-Posting**: Attackers can submit unexpected JSON properties (e.g. `IsPublished: true`) and overwrite entity state. 2. **Serialization Cycles**: Bidirectional EF Core relationships (`Course` -> `Sections` -> `Course`) trigger circular reference errors in JSON serializers. 3. **Information Disclosure**: Entities may contain

> sensitive data like `PasswordHash` or audit columns that should never be sent over the wire. 4. **API Contract Stability**: DTOs decouple internal database schema changes from public API contracts, preventing breaking changes for frontend clients.

---

11. Summary of Architectural Best Practices in SkillHub

1. **Dependency Inversion**: Dependencies point strictly inwards toward the core domain.
2. **Stateless Security**: JWT tokens combined with sliding refresh token rotation guarantee horizontal scalability.
3. **Defensive Deletion**: `DeleteBehavior.Restrict` prevents accidental cascading destruction of enterprise audit trails.
4. **Optimized Persistence**: Read operations leverage `.AsNoTracking()` and `.ProjectTo<T>()` to generate selective SQL and bypass change-tracking overhead.
5. **Contract-Driven Design**: Every business capability is governed by strongly typed interfaces and validated with DTOs.
6. **Dynamic Governance**: Dynamic policy providers empower administrators to reconfigure system privileges in real time.